

Actividad Integradora ToString - Colecciones

Programación III

Coordinador: Enferrel, Ariel
Profesor/es: Rigoni, Cinthia
Tutor/a:
Alumno/a: Lauk, Karen

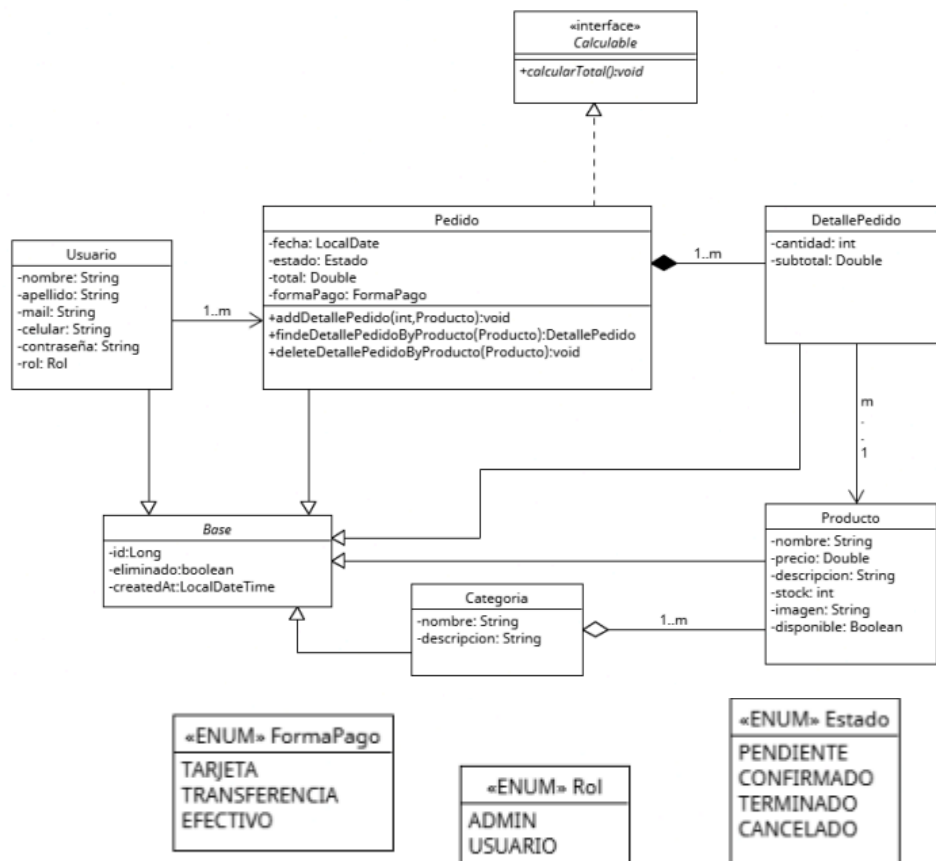
Objetivo

Desarrollar y procesar colecciones de objetos en Java, implementando toString() para una representación clara de los datos.

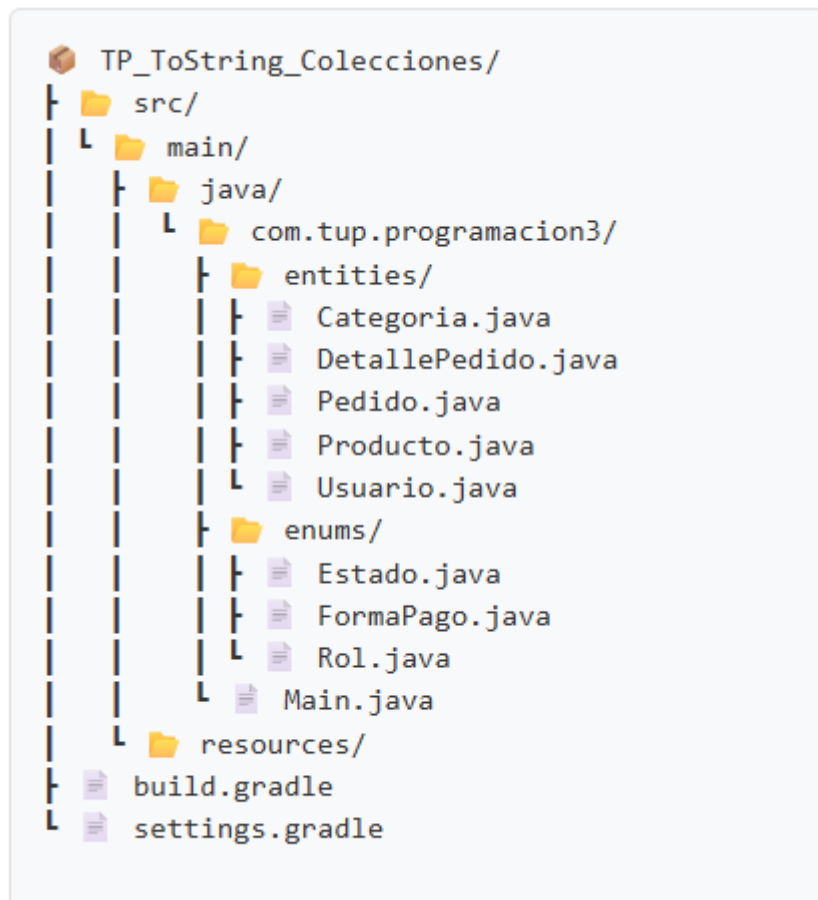
Caso Práctico

1. Desarrolle las clases con sus Relaciones correspondientes (utilice Colecciones de tipo Set).
2. En cada clase sobrescriba ToString, Equals y hashCode.
3. Deberá instanciar:
 - a. 2 Usuarios
 - b. 3 Pedidos (al menos 2 detalles pedido por cada uno)
 - c. 3 Categorías
 - d. 10 productos
4. En clase Main deberá de tener las instancias solicitadas en punto anterior y deberá utilizar toString para mostrar por consola un producto, el listado de productos cargados y los pedidos del usuario que más pedidos posea.
5. Instancie un producto nuevo donde el/los campos comparados en equals sean iguales, compare esa instancia con toda la colección. Mostrar resultados por pantalla.

UML



Estructuras



El proyecto fue desarrollado utilizando Gradle como herramienta de automatización y gestión de dependencias.

Organización de paquetes

- entities → contiene las clases principales del modelo de negocio (Usuario, Pedido, Producto, Categoria y DetallePedido).
- enums → contiene las enumeraciones utilizadas por el sistema (Rol, FormaPago y Estado).
- Main → clase principal encargada de crear las instancias solicitadas y ejecutar las pruebas requeridas por la consigna.
- resources → carpeta reservada para recursos externos del proyecto.
- build.gradle → archivo donde se configura Gradle y se administran las dependencias del proyecto.
- settings.gradle → archivo utilizado para definir la configuración general y el nombre del proyecto.

Respuestas:

1. Desarrollo de clases con sus relaciones:

Desarrolle las clases con sus relaciones utilizando colecciones exclusivas de tipo Set (mediante la implementación HashSet). En todas las entidades que poseen una relación de uno a muchos (1:N), la asociación se definió mediante un conjunto para asegurar de forma nativa la inexistencia de elementos duplicados en memoria.

→ Enumeraciones (Enums)

- ◆ Rol: ADMIN, USUARIO.
- ◆ FormaPago: TARJETA, TRANSFERENCIA, EFECTIVO.
- ◆ Estado: PENDIENTE, CONFIRMADO, TERMINADO, CANCELADO

→ Relaciones

◆ Usuario → 1:N → Pedido

Un usuario puede realizar múltiples pedidos. Por este motivo, la clase Usuario posee un atributo Set<Pedido> pedidos que almacena su historial de compras.

◆ Pedido → 1:N → DetallePedido

Cada pedido está compuesto por varias líneas de detalle. Esta relación se representa mediante el atributo Set<DetallePedido> detalles.

◆ Categoria → 1:N → Producto

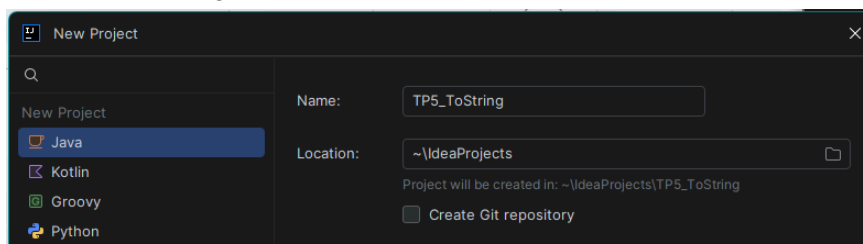
Una categoría puede agrupar varios productos del menú. Para ello, la clase Categoria contiene el atributo Set<Producto> productos.

◆ DetallePedido → N:1 → Producto

Cada detalle de pedido referencia a un único producto, indicando qué artículo fue solicitado dentro del pedido.

De esta manera, las relaciones uno a muchos (1:N) quedan modeladas mediante colecciones Set, garantizando la integridad de los datos y evitando registros duplicados.

Creamos el proyecto en IntelliJ IDE → Construí el sistema con Gradle



Creamos las enumeraciones

1. Rol

```
1 package com.tup.programacion3.enums;
2
3 public enum Rol { 6 usages
4     ADMIN, USUARIO 1 usage
5 }
```

2. FormaPago

```
1 package com.tup.programacion3.enums;
2
3 public enum FormaPago { 9 usages
4     TARJETA, TRANSFERENCIA, EFECTIVO 1 usage
5 }
```

3. Estado

```
1 package com.tup.programacion3.enums;
2
3 public enum Estado { 9 usages
4     PENDIENTE, CONFIRMADO, TERMINADO, CANCELADO 1 usage
5 }
6
```

Consigna práctica:

2. Cada clase va a estar sobrescriba ToString, Equals y hashCode

En cada clase se debe sobrescribir la representación de texto y el contrato de comparación.

- toString(): mostrar información legible por consola.
- equals(): comparar objetos según sus atributos relevantes.
- hashCode(): garantizar un comportamiento correcto dentro de colecciones Set.

Creación de las clases dentro de la carpeta Entities

1. Producto

```
1 package com.tup.programacion3.entities;
2 import java.util.Objects;
3
4 // INDICAMOS QUE HEREDA DE BASE
5 public class Producto { 36 usages
6     private Long id; 4 usages
7     private boolean eliminado; 3 usages
8     private String nombre; 7 usages
9     private Double precio; 7 usages
10    private String descripcion; 1 usage
11    private int stock; 4 usages
12    private String urlImagen; 1 usage
13    private boolean disponible; 4 usages
14
15    public Producto() {} no usages
16
17    public Producto(Long id, boolean eliminado, 11 usages
18        String nombre, Double precio, String descripcion,
19        int stock, String urlImagen, boolean disponible) {
20        this.id = id;
21        this.eliminado = eliminado;
22        this.nombre = nombre;
23        this.precio = precio;
24        this.descripcion = descripcion;
25        this.stock = stock;
26        this.urlImagen = urlImagen;
27        this.disponible = disponible;
28    }
29 }
```

```

30 // El contrato equals y hashCode para detectar duplicados por nombre y precio
31 @Override
32 public boolean equals(Object o) {
33     if (this == o) return true;
34     if (o == null || getClass() != o.getClass()) return false;
35     Producto producto = (Producto) o;
36     return Objects.equals(nombre, producto.nombre) && Objects.equals(precio, producto.precio);
37 }
38
39 @Override
40 public int hashCode() {
41     return Objects.hash(nombre, precio);
42 }
43
44 // GETTERS Y SETTERS
45 public Long getId() { return id; } no usages
46 public void setId(Long id) { this.id = id; } no usages
47
48 public boolean isEliminado() { return eliminado; } no usages
49 public void setEliminado(boolean eliminado) { this.eliminado = eliminado; } no usages
50
51 public String getNombre() { return nombre; } 2 usages
52 public void setNombre(String nombre) { this.nombre = nombre; } no usages
53
54 public Double getPrecio() { return precio; } 2 usages
55 public void setPrecio(Double precio) { this.precio = precio; } no usages
56
57 public int getStock() { return stock; } no usages
58 public void setStock(int stock) { this.stock = stock; } no usages
59
60 public boolean isDisponible() { return disponible; } no usages
61 public void setDisponible(boolean disponible) { this.disponible = disponible; } no usages
62
63 // TO STRING
64 @Override
65 public String toString() {
66     return "[ID: " + id + "] " + nombre +
67         " - $" + String.format("%.2f", precio) +
68         " | Stock: " + stock + " | Disponible: " + disponible;
69 }

```

Inden

2. Pedido

```
1 package com.tup.programacion3.entities;
2 import com.tup.programacion3.enums.Estado;
3 import com.tup.programacion3.enums.FormaPago;
4 import java.time.LocalDate;
5 import java.util.HashSet;
6 import java.util.Set;
7
8 public class Pedido { 10 usages
9     private Long id; 4 usages
10    private boolean eliminado; 4 usages
11    private LocalDate fecha; 5 usages
12    private Estado estado; 4 usages
13    private Double total; 5 usages
14    private FormaPago formaPago; 3 usages
15    private Set<DetallePedido> detalles; 8 usages
16
17    public Pedido() { no usages
18        this.fecha = LocalDate.now();
19        this.detalles = new HashSet<>();
20        this.total = 0.0;
21    }
22
23    public Pedido(Long id, boolean eliminado, Estado estado, FormaPago formaPago) { 3 usages
24        this.id = id;
25        this.eliminado = eliminado;
26        this.fecha = LocalDate.now();
27        this.estado = estado;
28        this.formaPago = formaPago;
29        this.detalles = new HashSet<>();
30        this.total = 0.0;
31    }
32
33    public void calcularTotal() { 2 usages
34        double suma = 0.0;
35        for (DetallePedido dp : detalles) {
36            suma += dp.getSubtotal();
37        }
38        this.total = suma;
39    }
40 }
```

```

41     public void addDetallePedido(int cantidad, Producto producto) { 6 usages
42         DetallePedido nuevoDetalle = new DetallePedido(cantidad, producto);
43         this.detalles.add(nuevoDetalle);
44         calcularTotal();
45     }
46
47     public DetallePedido findDetallePedidoByProducto(Producto producto) { 1 usage
48         for (DetallePedido dp : detalles) {
49             if (dp.getProducto() != null && dp.getProducto().equals(producto)) {
50                 return dp;
51             }
52         }
53         return null;
54     }
55
56     public void deleteDetallePedidoByProducto(Producto producto) { no usages
57         DetallePedido aEliminar = findDetallePedidoByProducto(producto);
58         if (aEliminar != null) {
59             this.detalles.remove(aEliminar);
60             calcularTotal();
61         }
62     }

```

```

// GETTERS Y SETTERS
public Long getId() { return id; } no usages
public void setId(Long id) { this.id = id; } no usages

public boolean isEliminado() { return eliminado; } no usages
public void setEliminado(boolean eliminado) { this.eliminado = eliminado; } no usages

public LocalDate getFecha() { return fecha; } no usages
public void setFecha(LocalDate fecha) { this.fecha = fecha; } no usages

public Estado getEstado() { return estado; } no usages
public void setEstado(Estado estado) { this.estado = estado; } no usages

public Double getTotal() { return total; } no usages

public FormaPago getFormaPago() { return formaPago; } no usages
public void setFormaPago(FormaPago formaPago) { this.formaPago = formaPago; } no usages

public Set<DetallePedido> getDetalles() { return detalles; } 1 usage

// TO STRING
@Override
public String toString() {
    return "Pedido{id=" + id + ", eliminado=" + eliminado +
        ", fecha=" + fecha + ", estado=" + estado +
        ", total=$" + total + ", items=" + detalles.size() + "}";
}
}

```

3. DetallePedido

```
1 package com.tup.programacion3.entities;
2
3 public class DetallePedido { 9 usages
4     private int cantidad; 5 usages
5     private Double subtotal; 4 usages
6     private Producto producto; 7 usages
7
8     public DetallePedido() {} no usages
9
10    public DetallePedido(int cantidad, Producto producto) { 1 usage
11        this.cantidad = cantidad;
12        this.producto = producto;
13        calcularSubtotal();
14    }
15
16    public void calcularSubtotal() { 3 usages
17        if (this.producto != null) {
18            this.subtotal = this.cantidad * this.producto.getPrecio();
19        } else {
20            this.subtotal = 0.0;
21        }
22    }
23
24    // GETTERS Y SETTERS
25    public int getCantidad() { return cantidad; } no usages
26    public void setCantidad(int cantidad) { no usages
27        this.cantidad = cantidad;
28        calcularSubtotal();
29    }
30
31    public Double getSubtotal() { return subtotal; } 1 usage
32
33    public Producto getProducto() { return producto; } 2 usages
34    public void setProducto(Producto producto) { no usages
35        this.producto = producto;
36        calcularSubtotal();
37    }
38
```

```
// TO STRING
@Override
public String toString() {
    return "DetallePedido{producto=" + (producto != null ? producto.getNombre() : "null") +
        ", cantidad=" + cantidad +
        ", subtotal=$" + subtotal + "}";
}
}
```


4. Usuario

```
1 package com.tup.programacion3.entities;
2 import com.tup.programacion3.enums.Rol;
3 import java.util.HashSet;
4 import java.util.Set;
5
6 public class Usuario { 5 usages
7     private Long id; 4 usages
8     private boolean eliminado; 3 usages
9     private String nombre; 3 usages
10    private String apellido; 3 usages
11    private String mail; 1 usage
12    private String celular; 1 usage
13    private String contrasenia; 1 usage
14    private Rol rol; 2 usages
15    private Set<Pedido> pedidos; 5 usages
16
17    public Usuario() { no usages
18        this.pedidos = new HashSet<>();
19    }
20
21    public Usuario(Long id, boolean eliminado, String nombre, 2 usages
22        String apellido, String mail, String celular,
23        String contrasenia, Rol rol) {
24        this.id = id;
25        this.eliminado = eliminado;
26        this.nombre = nombre;
27        this.apellido = apellido;
28        this.mail = mail;
29        this.celular = celular;
30        this.contrasenia = contrasenia;
31        this.rol = rol;
32        this.pedidos = new HashSet<>();
33    }
34
35    public void addPedido(Pedido pedido) { 3 usages
36        if (pedido != null) {
37            this.pedidos.add(pedido);
38        }
39    }
```

```

30 // GETTERS Y SETTERS
31 public String getNombre() { return nombre; } 1 usage
32 public void setNombre(String nombre) { this.nombre = nombre; } no usages
33
34 public String getApellido() { return apellido; } 1 usage
35 public void setApellido(String apellido) { this.apellido = apellido; } no usages
36
37 public String getMail() { return mail; } no usages
38 public void setMail(String mail) { this.mail = mail; } no usages
39
40 public String getCelular() { return celular; } no usages
41 public void setCelular(String celular) { this.celular = celular; } no usages
42
43 public String getContraseña() { return contraseña; } no usages
44 public void setContraseña(String contraseña) { this.contraseña = contraseña; } no usages
45
46 public Rol getRol() { return rol; } no usages
47 public void setRol(Rol rol) { this.rol = rol; } no usages
48
49 public Set<Pedido> getPedidos() { return pedidos; } 3 usages
50 public void setPedidos(Set<Pedido> pedidos) { this.pedidos = pedidos; } no usages
51
52 public void addPedido(Pedido pedido) { 3 usages
53     this.pedidos.add(pedido);
54 }
55
56 // TO STRING
57 @Override
58 public String toString() {
59     return "Usuario{" + super.toString() +
60         ", nombre='" + nombre +
61         "' + apellido +
62         "', mail='" + mail +
63         "', pedidos=" + pedidos.size() + "}";
64 }
65 }

```

```

40 // GETTERS Y SETTERS
41 public Long getId() { return id; } no usages
42 public void setId(Long id) { this.id = id; } no usages
43
44 public boolean isEliminado() { return eliminado; } no usages
45 public void setEliminado(boolean eliminado) { this.eliminado = eliminado; } no usages
46
47 public String getNombre() { return nombre; } 1 usage
48 public String getApellido() { return apellido; } 1 usage
49 public Set<Pedido> getPedidos() { return pedidos; } 3 usages
50
51 // TO STRING
52 public String toString() {
53     return "Usuario{id=" + id +
54         ", nombre='" + nombre +
55         "', apellido='" + apellido +
56         "', rol=" + rol +
57         ", pedidos=" + pedidos.size() + "}";
58 }
59 }

```

5. Categoria

```
1 package com.tup.programacion3.entities;
2
3 import java.util.HashSet;
4 import java.util.Set;
5
6 public class Categoria { 6 usages
7     private Long id; 4 usages
8     private boolean eliminado; 3 usages
9     private String nombre; 4 usages
10    private Set<Producto> productos; 5 usages
11
12    > public Categoria() { this.productos = new HashSet<>(); }
13
14    public Categoria(Long id, boolean eliminado, String nombre) { 3 usages
15        this.id = id;
16        this.eliminado = eliminado;
17        this.nombre = nombre;
18        this.productos = new HashSet<>();
19    }
20
21    public void addProducto(Producto producto) { 6 usages
22        if (producto != null) {
23            this.productos.add(producto);
24        }
25    }
26
27    // GETTERS Y SETTERS
28    public Long getId() { return id; } no usages
29    public void setId(Long id) { this.id = id; } no usages
30
31    public boolean isEliminado() { return eliminado; } no usages
32    public void setEliminado(boolean eliminado) { this.eliminado = eliminado; } no usages
33
34    public String getNombre() { return nombre; } no usages
35    public void setNombre(String nombre) { this.nombre = nombre; } no usages
36
37    public Set<Producto> getProductos() { return productos; } no usages
38    // TO STRING
39    public String toString() {
40        return "Categoria{id=" + id + ", nombre=" + nombre + ", productos=" + productos.size() + "}";
41    }
42
43 }
```

Consigna práctica:

3. Instancias obligatorias:

En la clase Main se realizaron las instancias requeridas:

Archivo Main

```
1 package com.tup.programacion3;
2 import com.tup.programacion3.enums.Estado;
3 import com.tup.programacion3.enums.FormaPago;
4 import com.tup.programacion3.enums.Rol;
5 import com.tup.programacion3.entities.*;
6
7 import java.util.HashSet;
8 import java.util.Set;
```

- Instancia de 3 categorías

Se inicializaron las categorías base para agrupar los elementos del menú (Hamburguesas, Pizzas y Bebidas).

```
public class Main {
    public static void main(String[] args) {
        // 1. Instanciar Categorías
        Categoria hamburguesas = new Categoria( id: 1L, eliminado: false, nombre: "Hamburguesas");
        Categoria pizzas = new Categoria( id: 2L, eliminado: false, nombre: "Pizzas");
        Categoria bebidas = new Categoria( id: 3L, eliminado: false, nombre: "Bebidas");
```

- Instanciamos 10 productos con sus atributos

Se creó un conjunto global indexado por hash (Set<Producto> catalogoProductos = new HashSet<>();) donde se insertaron manualmente 10 productos de cocina estables con sus respectivos identificadores, stock y precios.

```
// 2. Instanciar Catálogo de 10 Productos
Set<Producto> catalogoProductos = new HashSet<>();

Producto p1 = new Producto( id: 1L, eliminado: false,
    nombre: "Hamburguesa Triple", precio: 25000.0, descripcion: "Tres carnes y cheddar",
    stock: 15, urlImagen: "h1.jpg", disponible: true);
Producto p2 = new Producto( id: 2L, eliminado: false,
    nombre: "Pizza Calabresa", precio: 20500.0, descripcion: "Longaniza premium",
    stock: 10, urlImagen: "p1.jpg", disponible: true);
Producto p3 = new Producto( id: 3L, eliminado: false,
    nombre: "Bebida Cola", precio: 1500.0, descripcion: "Lata 354ml",
    stock: 50, urlImagen: "b1.jpg", disponible: true);
Producto p4 = new Producto( id: 4L, eliminado: false,
    nombre: "Papas Fritas", precio: 7500.0, descripcion: "Porción grande",
    stock: 30, urlImagen: "f1.jpg", disponible: true);
Producto p5 = new Producto( id: 5L, eliminado: false,
    nombre: "Pizza Mozzarella", precio: 18500.0, descripcion: "Mucha mozzarella",
    stock: 12, urlImagen: "p2.jpg", disponible: true);
Producto p6 = new Producto( id: 6L, eliminado: false,
    nombre: "Garden Ensalada", precio: 15500.0, descripcion: "Sana y fresca",
    stock: 8, urlImagen: "e1.jpg", disponible: true);
Producto p7 = new Producto( id: 7L, eliminado: false,
    nombre: "Sandwich Italiano", precio: 17000.0, descripcion: "En pan de campo",
    stock: 14, urlImagen: "s1.jpg", disponible: true);
Producto p8 = new Producto( id: 8L, eliminado: false,
    nombre: "Papas Fritas con Cheddar", precio: 9000.0, descripcion: "Papas con cheddar y bacon",
    stock: 25, urlImagen: "f2.jpg", disponible: true);
Producto p9 = new Producto( id: 9L, eliminado: false,
    nombre: "Hamburguesa Smash Supremo", precio: 22000.0, descripcion: "Dos carnes finitas",
    stock: 18, urlImagen: "h2.jpg", disponible: true);
Producto p10 = new Producto( id: 10L, eliminado: false,
    nombre: "Agua Mineral", precio: 1200.0, descripcion: "Con o sin gas",
    stock: 40, urlImagen: "b2.jpg", disponible: true);
```

```

catalogoProductos.add(p1); catalogoProductos.add(p2); catalogoProductos.add(p3);
catalogoProductos.add(p4); catalogoProductos.add(p5); catalogoProductos.add(p6);
catalogoProductos.add(p7); catalogoProductos.add(p8); catalogoProductos.add(p9);
catalogoProductos.add(p10);

// Asociar a categorías
hamburguesas.addProducto(p1); hamburguesas.addProducto(p9);
pizzas.addProducto(p2); pizzas.addProducto(p5);
bebidas.addProducto(p3); bebidas.addProducto(p10);

```

- Instanciar 2 usuarios

Se registraron dos usuarios en el sistema.

```

// 3. Instanciar Usuarios
Usuario u1 = new Usuario( id: 101L, eliminado: false, nombre: "Ana",
    apellido: "Martinez", mail: "ana@mail.com",
    celular: "112233", contrasenia: "123", Rol.USUARIO);
Usuario u2 = new Usuario( id: 102L, eliminado: false, nombre: "Carlos",
    apellido: "Gomez", mail: "carlos@mail.com",
    celular: "445566", contrasenia: "456", Rol.ADMIN);

```

- Instancia de 3 pedidos

Se distribuyeron tres pedidos entre los usuarios instanciados anteriormente, asegurando que cada orden contenga al menos dos objetos DetallePedido independientes para evaluar los algoritmos internos de cálculo matemático

```

// 4. Instanciar Pedidos
Pedido pedido1 = new Pedido( id: 1001L, eliminado: false, Estado.PENDIENTE, FormaPago.EFECTIVO);
pedido1.addDetallePedido( cantidad: 2, p1);
pedido1.addDetallePedido( cantidad: 1, p4);

Pedido pedido2 = new Pedido( id: 1002L, eliminado: false, Estado.CONFIRMADO, FormaPago.TARJETA);
pedido2.addDetallePedido( cantidad: 1, p2);
pedido2.addDetallePedido( cantidad: 2, p3);

Pedido pedido3 = new Pedido( id: 1003L, eliminado: false, Estado.TERMINADO, FormaPago.TRANSFERENCIA);
pedido3.addDetallePedido( cantidad: 1, p8);
pedido3.addDetallePedido( cantidad: 1, p9);

u1.addPedido(pedido2);
u1.addPedido(pedido3);
u2.addPedido(pedido1);

```

4. Mostrar resultados

```
System.out.println("INFORME GENERAL\n");
System.out.println(" A- MUESTRA DE UN PRODUCTO INDIVIDUAL:");
System.out.println(p1 + "\n");

System.out.println(" B- LISTADO COMPLETO DEL CATÁLOGO DE PRODUCTOS:");
for (Producto prod : catalogoProductos) {
    System.out.println(prod);
}
System.out.println();
```

```
System.out.println(" C- DETALLE DE PEDIDOS DEL USUARIO CON MÁS COMPRAS:");
Usuario masActivo = (u1.getPedidos().size() > u2.getPedidos().size()) ? u1 : u2;
System.out.println("Cliente más activo: " + masActivo.getNombre() + " " + masActivo.getApellido());
for (Pedido p : masActivo.getPedidos()) {
    System.out.println("----- ");
    System.out.println(p);
    System.out.println("    Detalles de este pedido:");
    for (DetallePedido dp : p.getDetalles()) {
        System.out.println("        -> " + dp);
    }
}
System.out.println("----- \n");
```

❖ Mostrar un producto usando toString():

Invocando el método sobre la instancia seleccionada, se visualiza la información limpia del producto individual en una única línea de texto formateada.

❖ Mostrar listado de productos

Al imprimir el catálogo completo de 10 productos almacenados en el HashSet, se constata que el orden de salida en la terminal es aleatorio y no respeta la secuencia física de inserción en el código. Esto valida empíricamente el comportamiento del Set, demostrando el funcionamiento real de indexación por tablas de hash.

❖ Usuario con más pedidos

El sistema analizó las colecciones en memoria, identificó de forma automática que la cliente más activa es Ana Martínez (con 2 pedidos en su historial) y procedió a desplegar en detalle el desglose de sus órdenes (Pedido 1002 y Pedido 1003). La terminal refleja con total exactitud matemática el cálculo de los subtotales por ítem (cantidad * precio) y la sumatoria final del importe total de cada pedido.

5. Pruebas de Comparación y Unicidad

Se creó un producto con los mismos atributos utilizados en equals() que otro producto ya existente.

- Test de duplicados mediante equals y set

Para verificar la robustez del contrato de objetos, se instanció en el Main un nuevo producto clonado ("Hamburguesa Triple" con un precio exacto de \$25000,00), simulando una operación de carga de datos redundante en el sistema. Se procedió a evaluar su comportamiento contra la colección existente mediante los métodos .contains() y .add().

```
// 5. TEST DE DUPLICADOS MEDIANTE EQUALS Y SET
System.out.println("PRUEBA DE COMPARACIÓN Y UNICIDAD");
Producto productoClonado = new Producto( id: 999L, eliminado: false,
    nombre: "Hamburguesa Triple", precio: 25000.0, descripción: "Copia fraudulenta",
    stock: 1, urlImagen: "falso.jpg", disponible: true);
System.out.println("Producto clonado creado para evaluar: " + productoClonado.getNombre() + " ($" + productoClonado.getPrecio() + ")");

boolean existe = catalogoProductos.contains(productoClonado);
System.out.println("¿El sistema detecta el clon por compartir Nombre y Precio?: " + (existe ? "SÍ, YA EXISTE" : "NO EXISTE"));

boolean agregado = catalogoProductos.add(productoClonado);
System.out.println("¿El Set permitió añadir el producto repetido?: " + (agregado ? "SÍ (Error)" : "NO (Rechazado correctamente)"));
System.out.println("Cantidad total de productos finales en la colección: " + catalogoProductos.size());
}
```

Evaluación de resultados

Se realizó una prueba de validación creando una nueva instancia de Producto con los mismos atributos utilizados en los métodos equals() y hashCode() que un producto ya existente en la colección.

Los resultados obtenidos fueron los siguientes:

1. El sistema identificó correctamente que ambos objetos eran equivalentes.
2. El método equals() devolvió true, confirmando la igualdad entre las instancias.
3. La colección de tipo Set impidió la incorporación del elemento duplicado.
4. La cantidad total de productos almacenados permaneció sin cambios, verificando el correcto funcionamiento de hashCode() y equals().
5. El programa ejecutó todas las pruebas satisfactoriamente y finalizó su ejecución sin errores mediante Gradle, obteniendo el código de salida 0 (Process finished with exit code 0).

INFORME GENERAL

A- MUESTRA DE UN PRODUCTO INDIVIDUAL:

[ID: 1] Hamburguesa Triple - \$25000,00 | Stock: 15 | Disponible: true

B- LISTADO COMPLETO DEL CATÁLOGO DE PRODUCTOS:

[ID: 4] Papas Fritas - \$7500,00 | Stock: 30 | Disponible: true
[ID: 5] Pizza Mozzarella - \$18500,00 | Stock: 12 | Disponible: true
[ID: 8] Papas Fritas con Cheddar - \$9000,00 | Stock: 25 | Disponible: true
[ID: 9] Hamburguesa Smash Supremo - \$22000,00 | Stock: 18 | Disponible: true
[ID: 1] Hamburguesa Triple - \$25000,00 | Stock: 15 | Disponible: true
[ID: 3] Bebida Cola - \$1500,00 | Stock: 50 | Disponible: true
[ID: 6] Garden Ensalada - \$15500,00 | Stock: 8 | Disponible: true
[ID: 7] Sandwich Italiano - \$17000,00 | Stock: 14 | Disponible: true
[ID: 2] Pizza Calabresa - \$20500,00 | Stock: 10 | Disponible: true
[ID: 10] Agua Mineral - \$1200,00 | Stock: 40 | Disponible: true

C- DETALLE DE PEDIDOS DEL USUARIO CON MÁS COMPRAS:

Cliente más activo: Ana Martínez

Pedido{id=1003, eliminado=false, fecha=2026-06-01, estado=TERMINADO, total=\$31000.0, items=2}

Detalles de este pedido:

-> DetallePedido{producto=Papas Fritas con Cheddar, cantidad=1, subtotal=\$9000.0}
-> DetallePedido{producto=Hamburguesa Smash Supremo, cantidad=1, subtotal=\$22000.0}

Pedido{id=1002, eliminado=false, fecha=2026-06-01, estado=CONFIRMADO, total=\$23500.0, items=2}

Detalles de este pedido:

-> DetallePedido{producto=Pizza Calabresa, cantidad=1, subtotal=\$20500.0}
-> DetallePedido{producto=Bebida Cola, cantidad=2, subtotal=\$3000.0}

PRUEBA DE COMPARACIÓN Y UNICIDAD

Producto clonado creado para evaluar: Hamburguesa Triple (\$25000.0)

¿El sistema detecta el clon por compartir Nombre y Precio?: SÍ, YA EXISTE

¿El Set permitió añadir el producto repetido?: NO (Rechazado correctamente)

Cantidad total de productos finales en la colección: 10

Process finished with exit code 0

Conclusión

La resolución y posterior reestructuración de esta actividad integradora permitió comprender la enorme importancia de dominar el Java Collections Framework. Queda demostrado que el control de la integridad y unicidad de los datos no requiere de la escritura de bucles condicionales complejos o validaciones manuales costosas a lo largo del código. Al implementar de manera precisa el contrato fundamental de equals() y hashCode(), el propio entorno de ejecución de Java se encarga de proteger las reglas del negocio de forma nativa en la memoria RAM del servidor.